

Objectives

Linked List

Linked List ADT

Applications of Linked List

Linked List – Java collections

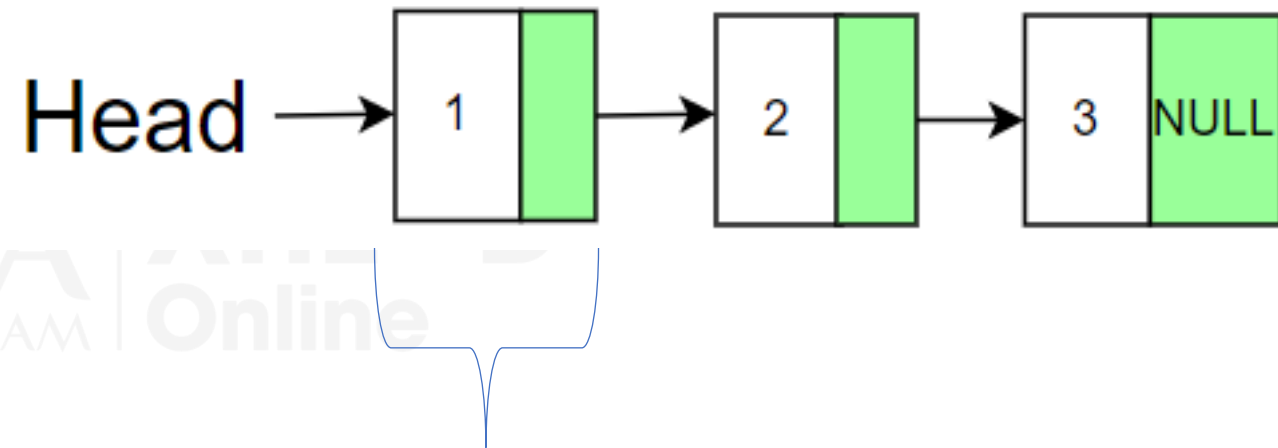
Linked List

- Linear Data Structure.
- Stores collection of items(list of items).
- Follows a linked representation.(successive items are not stored sequentially in adjacent locations as in arrays, items may be placed any where in the memory.)
- Each element of the list stores the address of next element for accessing list in correct order.

Linked List

Head - Stores the address of the first node.

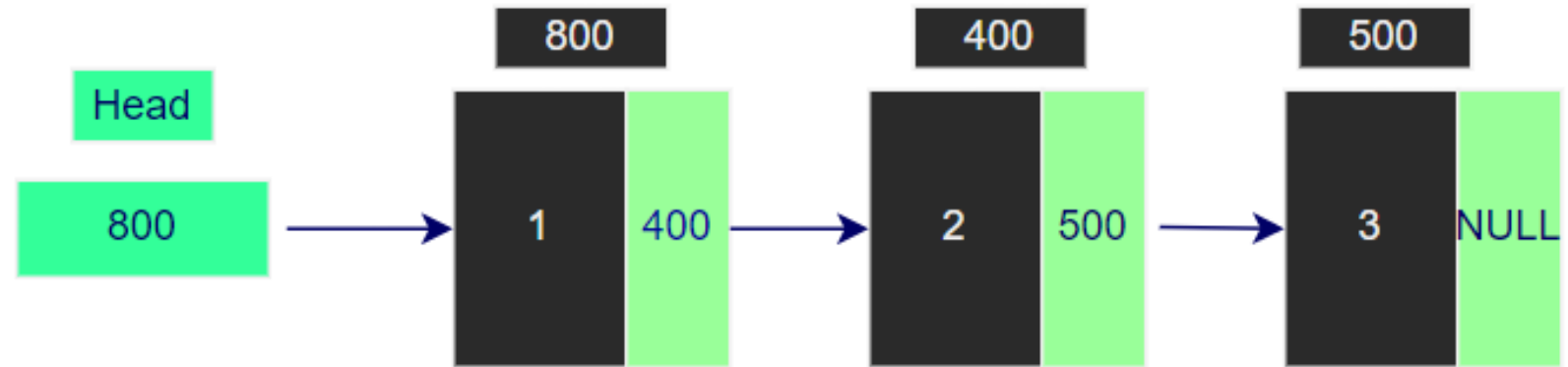
Node - Stores data and address to next node, which is used for accessing next data item. Address part of the last node is NULL. It marks the end of the list.



Node in a list, with a data part and next address part.

Linked List

Head variable stores the address of first node, 800. First node contains data value, 1 and address to next node ie, 400.



Linked List - Properties

- Accessed using a Head/Start pointer.
- Successive elements are connected by pointers.
- Next address field of last element points to NULL.
- Can grow or shrink dynamically.
- Elements are sequentially accessed.
- Random access is not possible.

Linked List -ADT

The following operations are supported by linked list:

- **Insert_to_list(item,pos)** //Insert item to a list in position **pos**.
If $pos==0$ then
 insert at the beginning of the list.
else If $pos==n$ then // n is number of nodes in the list,
 insert to end of the list.
else
 insert in position **pos**
- **item Del_from_list(pos)**
Delete an item at position pos and return the value deleted.
- **Traverse()** - Visit each node and do some action say, print the value.

Linked List -Applications

1. Implement data structures like Stack, Queue, Tree, Graph, Sparse Matrix
2. Memory Management in Operating System.
3. Process management in Operating System.



Linked List class Java

Linked List is a part of the Collection framework present in java.util package.

```
LinkedList<Integer> L=new LinkedList<Integer>(); //create linked list of  
//Integers
```

```
L.add(100); //add to end of list
```

```
L.add(200);
```

```
L.addFirst(1); //add to beginning of the list
```

Linked List Methods

`System.out.println(L.getFirst()); //return first element`

`System.out.println(L.getLast()); //return last element`

`System.out.println(L.removeFirst()); //removes first element`

`System.out.println(L.removeLast()); //removes last element`

References

Data Structures & Problem Solving Using Java, Mark Allen Weiss
Fundamentals of Data Structures. Ellis Horowitz and Sartaj Sahni



Objectives

Linked List Implementation

Linked List Operations

insert

delete

traverse



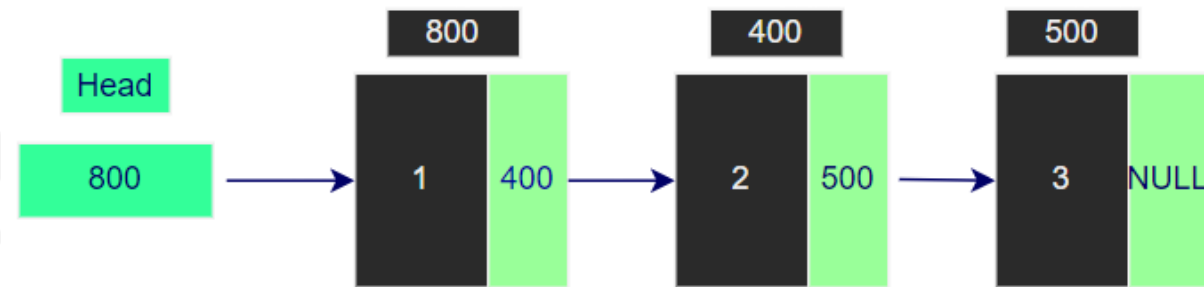
Linked List

- List consists of Nodes – design a class for a Node

```
class Node{  
    int data;  
    Node next;  
}
```

- List is collection of Nodes – design a class for a List

```
class LList{  
    private Node Head;  
}
```



Linked List – class Node

```
class Node{  
    private int data;  
    private Node next;  
    public Node() {data=0;next=null;}  
    public Node(int d, Node n) {data=d;next=n;}  
    public void print() {System.out.println(data);}  
    public void setNext(Node p) {next=p;}  
    public Node getNext() {return next;}  
    public void setData(int d) {data=d;}  
    public int getData(){return data;}  
}
```

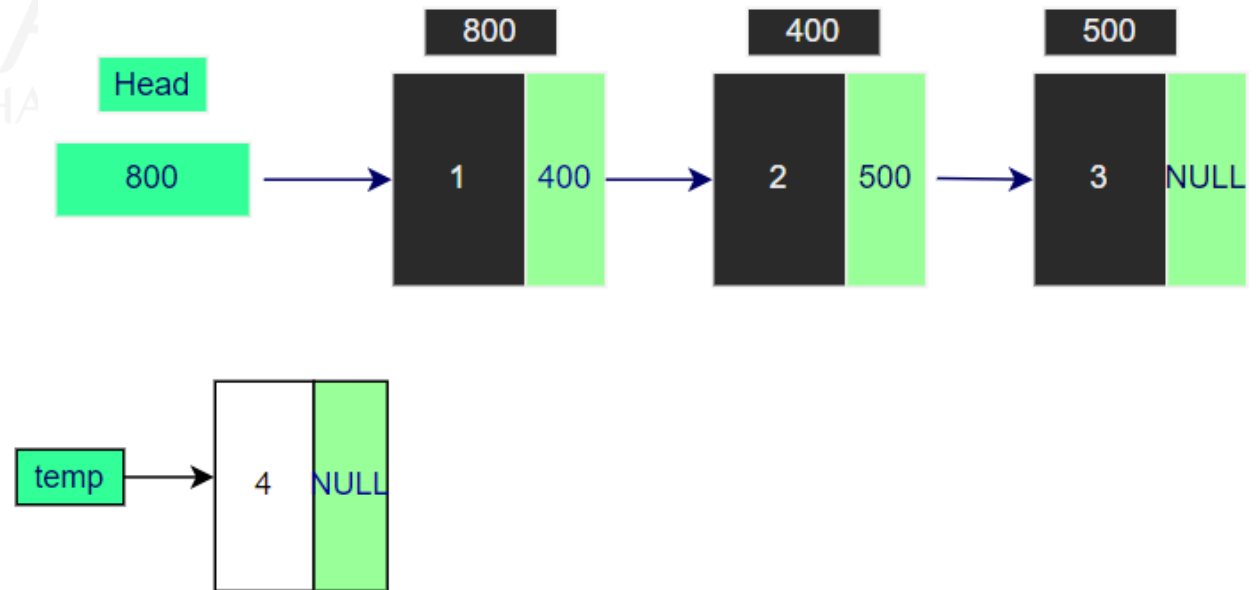
Linked List – class Linked List

Place your
Webcam Video here
Size 38%

```
class LList{  
    private Node head;  
    public LList(){head=null;}  
  
    public void Insert_to_list(int item,int pos) {}  
}
```

Linked List – insert

```
public void insert_to_list(int item,int pos) {  
    if(pos==0){        //pos 0, insert to head  
        Node temp=new Node(item,null);  
        temp.setNext(head);  
        head=temp;  
    }  
}
```



Linked List – insert

else if(pos==size){ //traverse till the last node of the list and insert at end

Node t=head;

while(t.getNext()!=null){

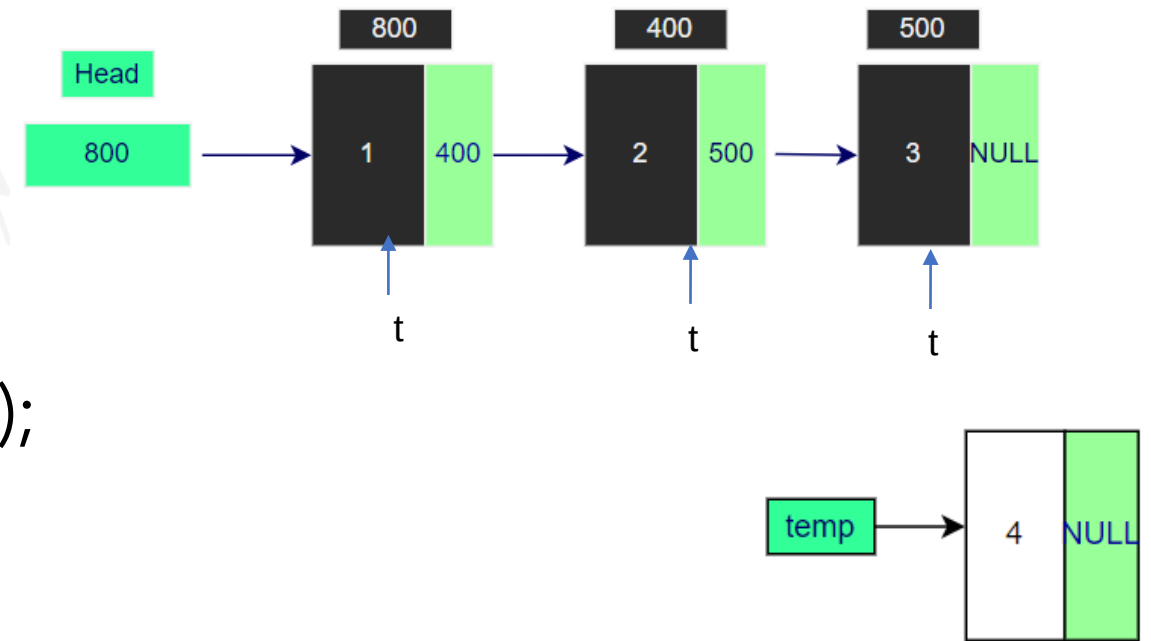
t=t.getNext();

}

Node temp=new Node(item,null);

t.setNext(temp);

}



Linked List – insert

```
else{
```

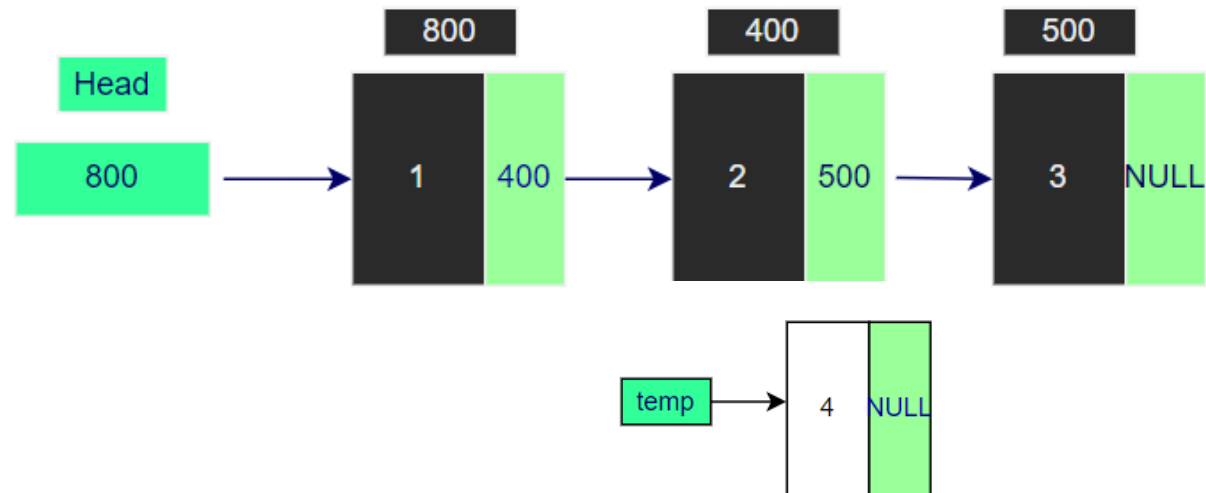
```
    Node t=head;
```

```
    while(t.getNext()!=null && pos>1){
```

```
        t=t.getNext(); pos--;
```

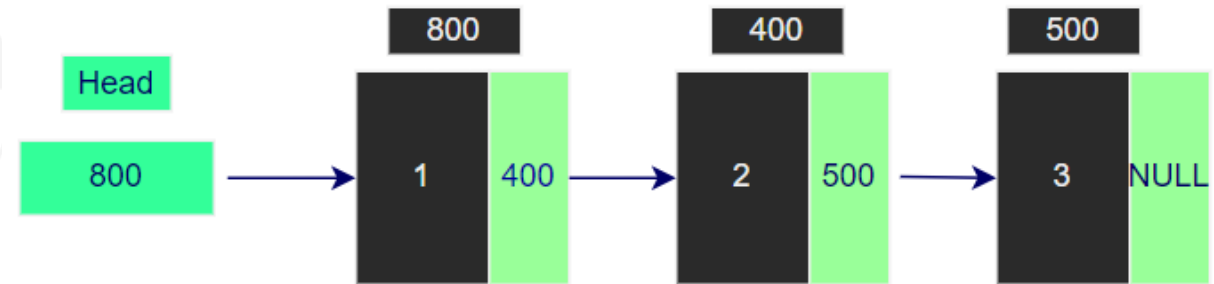
```
    Node temp= new Node(item,t.getNext());  
    t.setNext(newnode);
```

```
    } }
```



Linked List – delete

```
public int del_from_list(int p){  
    if(head==null){  
        System.out.println("empty list");  
        return -1;  
    }else if(p==0){  
        int val=head.getData();  
        head=head.getNext();  
        return val;  
    }  
}
```



Linked List – delete

```
else if(p==size){ //size of the list
```

```
    Node t=head;
```

```
    Node prev=null;
```

```
    while(t.getNext()!=null) {
```

```
        prev=t;
```

```
        t=t.getNext();    }
```

```
    int val=t.getData();
```

```
    prev.setNext(null);
```

```
    return val;
```

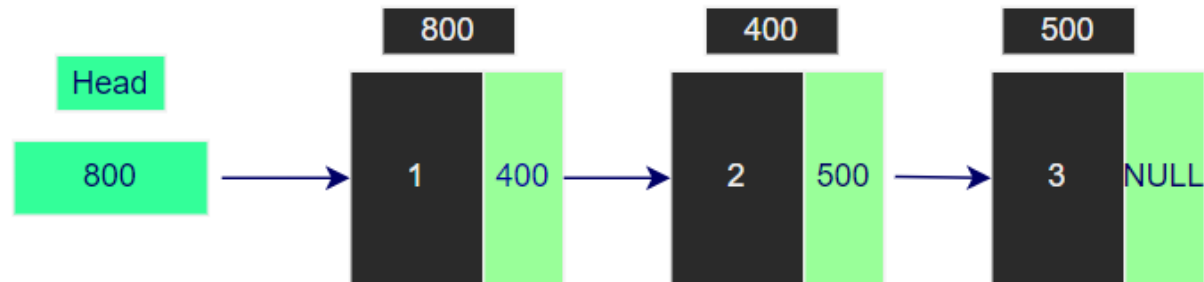
```
}
```



Linked List – delete

```
else{
```

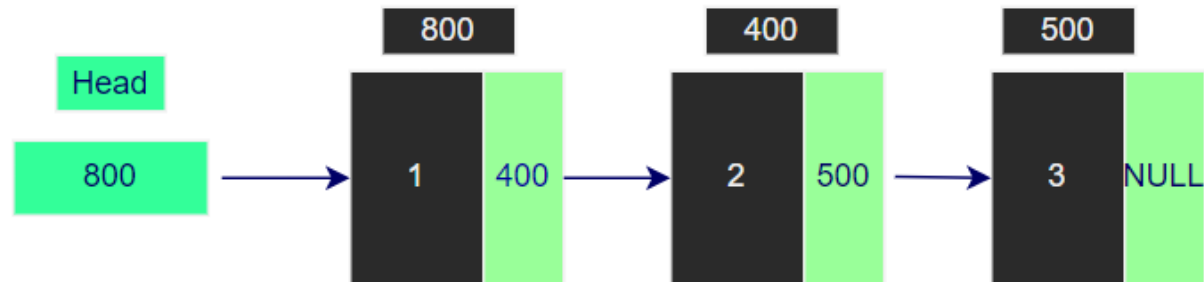
```
    Node temp=head;    Node prev=head;  
    while(temp.getNext()!=null && p>1){  
        prev=temp;  
        temp=temp.getNext(); p--;  
    }
```



Linked List – delete

```
int data=temp.getData();  
if(prev==temp)  
    head=head.getNext();  
else{  
    prev.setNext(temp.getNext());  
    return data;  
}
```

```
}  
}  
}
```



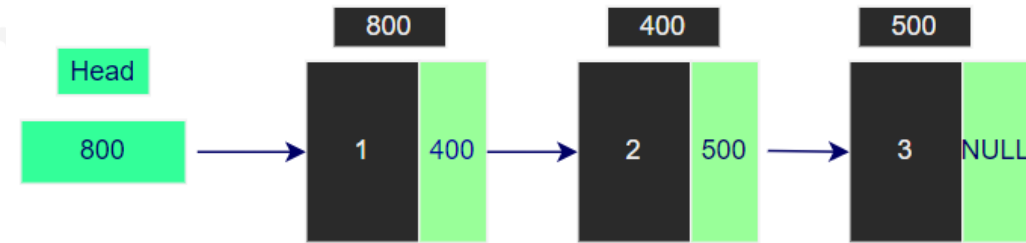
Linked List – Traversal

```
void print(){
```

```
    Node temp=head;  
    while(temp!=null){
```

```
        System.out.println(temp.getData());  
        temp=temp.getNext();  
    }
```

```
}
```



Summary

We have learnt how to implement a linked list and perform different operations on the linked list.



References

Data Structures & Problem Solving Using Java, Mark Allen Weiss
Fundamentals of Data Structures. Ellis Horowitz and Sartaj Sahni



Objectives

Loop Detection in Linked List



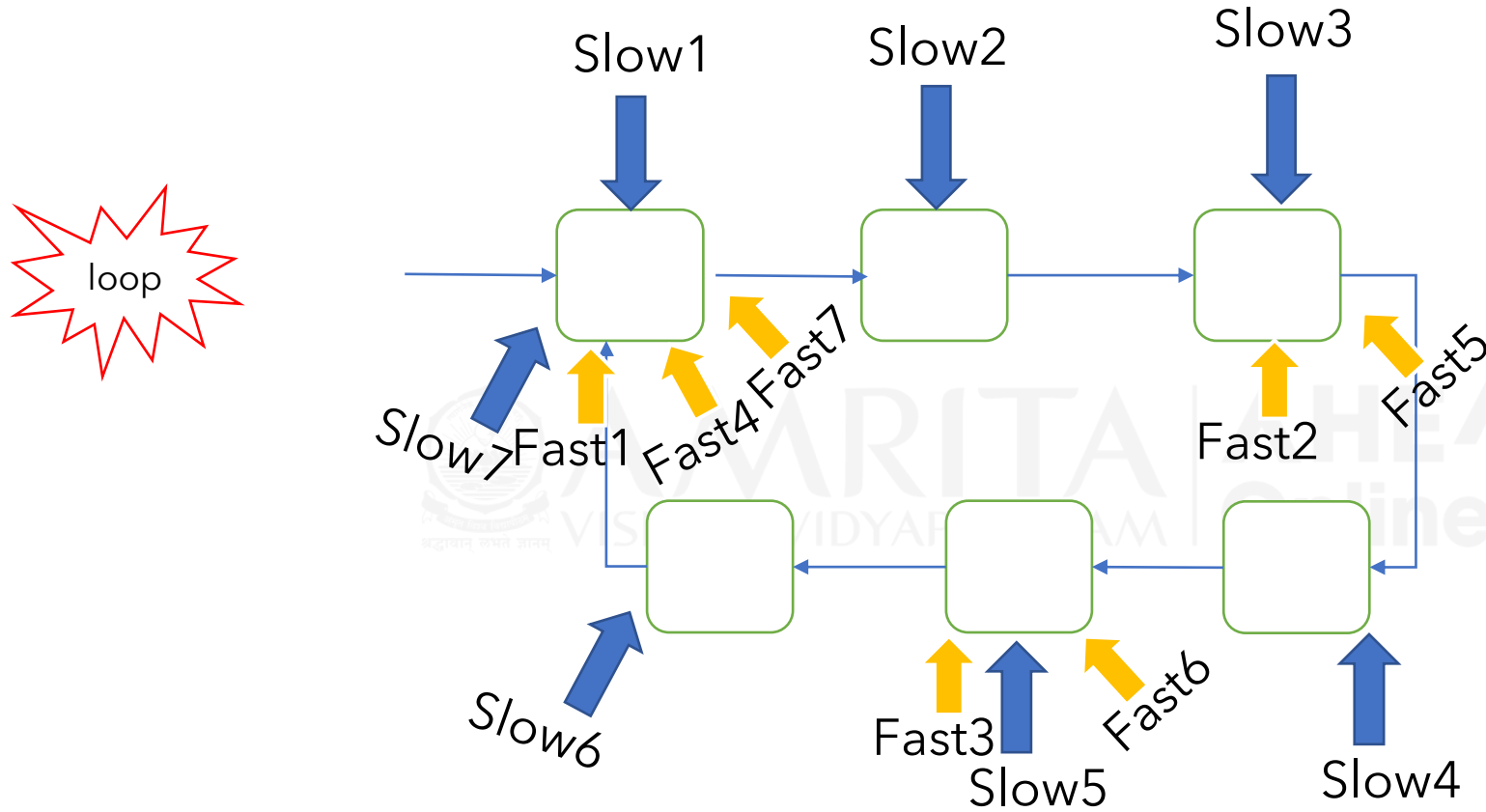
Loop Detection in Linked List

Floyd's Cycle-Finding Algorithm

1. Traverse the linked list using 2 references, Slow and Fast.
2. Fast reference moves by skipping one node in the list.
3. Slow reference moves without skipping, just moves to next node.
4. If the two references meet at a node then they are moving in a cycle and a loop exists in the linked list.



Loop Detection in Linked List – How it works?



Loop Detection in Linked List – Implementation

```
public static boolean loopCheck(LList lst) {  
    Node slowref=null,fastref=null;  
    slowref=lst.getHead();  
    fastref=slowref;  
    while(slowref!=null && fastref!=null && fastref.getNext()!=null) {  
        slowref=slowref.getNext();  
        fastref=(fastref.getNext()).getNext();  
        if(slowref==fastref)  
            return true;  
    }  
    return false;  
}
```

Summary

Determine a loop in a list.



Reference

Data Structures and Algorithms made easy - Narasimha Karumanchi

